

电子科技大学中山学院

课程设计说明书

学生信息					
系别	计算机学院	专业	计算机科学与技术		
班级	22 计科 B 班	姓名	杨永正	学号	2022030101082
课程设计信息					
课程名称	网络协议及应用开发				
课程设计题目	基于鸿蒙的智慧农业大棚监控系统				
课程设计时间	2024-2025 学年第 2 学期第 1-16 周				
小组情况	1 人 1 组			指导教师	陈浩广
批改情况					
成绩		评阅教师	陈浩广	批改时间	

2024—2025 学年第 2 学期

目 录

1. 课程设计内容	1
2. 课程设计目的	1
3. 背景知识	1
4. 工具/准备工作	2
5. 设计步骤与方法	2
5.1. 步骤 1: LED 灯控制设计	2
5.2. 步骤 2: KEY 按键检测设计	3
5.3. 步骤 3: 温湿度传感器设计	5
5.4. 步骤 4: 光敏传感检测设计	7
5.5. 步骤 5: wifi 通信模块程序设计	9
5.6. 步骤 6: 主程序的设计流程	10
6. 设计结果及分析	12
6.1. 项目实施过程	12
6.2. 系统框架	12
6.3. 遇到的问题及解决方案	12
6.4. 界面效果图	13
7. 设计结论	14
7.1. 设计得到的结论	14
7.2. 课程设计实现的效果说明	14
8. 问题及心得体会	15
8.1. 遇到的问题	15
8.2. 心得体会	16
9. 对本课程设计的改进建议	16
9.1. 课程内容方面	16
9.2. 实验环境方面	17
10. 参考文献	17

课程设计报告

1. 课程设计内容

本项目基于鸿蒙操作系统（OpenHarmony）和 BearPi-HM Nano 开发板，设计并实现了一个完整的智慧农业大棚监控系统，主要功能包括：

1. 按键检测模块

通过外部中断检测按键按下事件。

按键触发后，LED 状态切换。

2. 环境数据采集

实时监测大棚内的温度、湿度、光照强度。

使用 E53_IA1 扩展板的 SHT30 温湿度传感器和 BH1750 光敏传感器。

3. 设备控制

远程控制补光灯（基于光照强度自动/手动调节）。

远程控制通风机（基于温度阈值自动启停）。

4. 数据通信

通过 WiFi 模块连接路由器，实现 TCP/IP 网络通信。

使用 MQTT 协议将数据上报至华为 IoT 平台，并接收平台下发的控制指令。

5. 本地交互

通过板载按键（GPIO 中断）切换 LED 灯状态（开/关/闪烁）。

本地串口日志输出，便于调试。

2. 课程设计目的

考察学生对课程的掌握程度，以及学生实际的动手能力，编程能力。

3. 背景知识

- 1) 了解掌握“BearPi-HM Nano”开发板的硬件资源。
- 2) 了解掌握 ninja 与 gn 工程管理工作，熟悉 openharmony 操作系统。
- 2) 熟练掌握 vscode 的编写和调试程序的方法。
- 3) 熟练使用 HiBurn 下载软件烧写程序。
- 4) 熟悉“BearPi-HM Nano”开发板的 GPIO、中断、PWM、KV 存储等模块的

工作原理和应用方法。

5) 学会查看开发板的原理图，并分析 LED 灯、按键和蜂鸣器、光敏电阻、温湿度传感器、蓝牙模块的控制电路。

6) 熟悉 WIFI 模块的使用方法。

7) 掌握 TCP/UDP 协议，能使用套接字进行应用编程。

4. 工具/准备工作

硬件：

PC 机一台

“BearPi-HM Nano” 开发板一套

软件：

Windows10 操作系统

Ubuntu20.04 操作系统

HiBurn

MobaXterm

5. 设计步骤与方法

5.1. 步骤 1：LED 灯控制设计

目标：通过 GPIO 控制 LED 灯的状态（开、关、闪烁）。

实现方法：

1. GPIO 初始化：

初始化 GPIO 引脚（如 GPIO_2），设置为输出模式。

通过 IoTgpioInit 函数初始化指定的 GPIO 引脚。

使用 IoTgpioSetDir 函数将 GPIO 方向设置为输出模式，确保可以控制 LED 灯的状态。

```
//初始化GPIO_2
IoTgpioInit(LED_GPIO);
//设置GPIO_2为输出模式
IoTgpioSetDir(LED_GPIO, IOT_GPIO_DIR_OUT);
```

2. LED 状态控制：

定义一个全局变量 g_LedState，用于存储 LED 的状态（LED_ON、LED_OFF、LED_BLINK）。

在任务函数 LedTask 中，根据 g_LedState 的状态，通过 IoTgpioSetOutputVal

函数控制 GPIO 输出高电平或低电平。

```
switch (g_LedState) {
    case LED_ON:
        //设置GPIO_2输出高电平点亮LED灯
        IoTgpioSetOutputVal(LED_GPIO, 1);
        osDelay(100);
        break;
    case LED_OFF:
        //设置GPIO_2输出低电平熄灭LED灯
        IoTgpioSetOutputVal(LED_GPIO, 0);
        osDelay(100);
        break;
    case LED_BLINK:
        //设置GPIO_2交替输出高低电平，使LED闪烁
        IoTgpioSetOutputVal(LED_GPIO, 0);
        osDelay(100);
        IoTgpioSetOutputVal(LED_GPIO, 1);
        osDelay(100);
        break;
    default:
        osDelay(100);
        break;
}
```

3. 任务创建:

创建一个线程任务 LedTask，用于控制 LED 灯的状态。

设置线程的属性，如名称、优先级和栈大小。

```
osThreadAttr_t attr;

attr.name = "LedTask";
attr.attr_bits = 0U;
attr.cb_mem = NULL;
attr.cb_size = 0U;
attr.stack_mem = NULL;
attr.stack_size = 512;
attr.priority = osPriorityNormal;

if (osThreadNew((osThreadFunc_t)LedTask, NULL, &attr) == NULL)
{
    printf("Falied to create LedTask!\n");
}
```

5.2. 步骤 2: KEY 按键检测设计

目标：通过外部中断检测按键按下事件，并切换 LED 灯的状态。

实现方法：

1. 按键 GPIO 初始化：

初始化按键对应的 GPIO 引脚（如 GPIO_11 和 GPIO_12），设置为输入模式。
配置为上拉模式

```
//按键F1 GPIO初始化
IoTGpioInit(Button_F1_GPIO);
IoTGpioSetFunc(Button_F1_GPIO, IOT_GPIO_FUNC_GPIO_11_GPIO);

//按键F2 GPIO初始化
IoTGpioInit(Button_F2_GPIO);
IoTGpioSetFunc(Button_F2_GPIO, IOT_GPIO_FUNC_GPIO_12_GPIO);
```

使用 IoTGpioSetFunc 函数将 GPIO 功能设置为普通 GPIO。

使用 IoTGpioSetDir 函数将 GPIO 方向设置为输入模式。

使用 IoTGpioSetPull 函数将 GPIO 设置为上拉模式。

2. 中断回调函数注册：

注册按键中断回调函数，当按键按下时触发中断。

在回调函数中设置全局变量 g_LedState，以改变 LED 灯的状态。

使用 IoTGpioRegisterIsrFunc 函数注册中断回调函数，设置中断类型为下降沿触发。

```
//注册按键F1 的中断回调函数
IoTGpioRegisterIsrFunc(Button_F1_GPIO, IOT_INT_TYPE_EDGE,
| | | | | IOT_GPIO_EDGE_FALL_LEVEL_LOW, F1_Pressed, NULL);

//注册按键F2 的中断回调函数
IoTGpioRegisterIsrFunc(Button_F2_GPIO, IOT_INT_TYPE_EDGE,
| | | | | IOT_GPIO_EDGE_FALL_LEVEL_LOW, F2_Pressed, NULL);
```

3. 任务创建：

创建一个线程任务 LedTask，用于处理按键事件并控制 LED 灯的状态。

设置线程的属性，如名称、优先级和栈大小。

```

osThreadAttr_t attr;

attr.name = "LedTask";
attr.attr_bits = 0U;
attr.cb_mem = NULL;
attr.cb_size = 0U;
attr.stack_mem = NULL;
attr.stack_size = 512;
attr.priority = osPriorityNormal;

if (osThreadNew((osThreadFunc_t)LedTask, NULL, &attr) == NULL)
{
    printf("Falied to create LedTask!\n");
}

```

5.3. 步骤 3：温湿度传感器设计

目标：通过 SHT30 温湿度传感器采集大棚内的温度和湿度数据。

实现方法：

1. 传感器初始化：

初始化 I2C 接口，设置 I2C 通信速率（如 400kbps）。

```
IoTI2cInit(IOT_I2C_IDX_1, 400000); /* baudrate: 400kbps */
```

发送复位命令和测量周期设置命令到 SHT30 传感器。

```

/*****
* 函数名称: SHT30_reset
* 说    明: SHT30复位
* 参    数: 无
* 返 回 值: 无
*****/
void SHT30_reset(void)
{
    uint8_t send_data[2] = { 0x30, 0xA2 };
    IoTI2cWrite(IOT_I2C_IDX_1, (SHT30_Addr<<1) | 0x00, send_data, 2);
}

```

2. 数据采集：

向 SHT30 发送读取数据命令，读取温度和湿度的原始数据。

使用 CRC 校验验证数据的正确性。

如果校验通过，将数据转换为温度和湿度值，并存储到指定的变量中。


```

IoTI2cWriteread(IOT_I2C_IDX_1, (SHT30_Addr<<1)|0x00, &sht30_i2c_data);

data[0] = SHT3X_Data_Buffer[0];
data[1] = SHT3X_Data_Buffer[1];
data[2] = SHT3X_Data_Buffer[2];

tmp = SHT3x_CheckCrc(data, 2, data[2]);
if (!tmp)
{
    dat = ((uint16_t)data[0] << 8) | data[1];
    ReadData->Temperature = SHT3x_CalcTemperatureC(dat);
}

data[0] = SHT3X_Data_Buffer[3];
data[1] = SHT3X_Data_Buffer[4];
data[2] = SHT3X_Data_Buffer[5];

tmp = SHT3x_CheckCrc(data, 2, data[2]);
if (!tmp)
{
    dat = ((uint16_t)data[0] << 8) | data[1];
    ReadData->Humidity = SHT3x_CalcRH(dat);
}

```

3. 温度和湿度计:

1) 温度计算公式: $\text{温度} = 175 * (\text{原始数据} / 65535) - 45$ 。

```

/*****
* 函数名称: SHT3x_CalcTemperatureC
* 说    明: 温度计算
* 参    数: u16sT: 读取到的温度原始数据
* 返 回 值: 计算后的温度数据
*****/
static float SHT3x_CalcTemperatureC(uint16_t u16sT)
{
    float temperatureC = 0;

    u16sT &= ~0x0003;

    temperatureC = (175 * (float)u16sT / 65535 - 45);

    return temperatureC;
}

```

2) 湿度计算公式: $\text{湿度} = 100 * (\text{原始数据} / 65535)$ 。

```

/*****
* 函数名称: SHT3x_CalcRH
* 说    明: 湿度计算
* 参    数: u16sRH: 读取到的湿度原始数据
* 返 回 值: 计算后的湿度数据
*****/
static float SHT3x_CalcRH(uint16_t u16sRH)
{
    float humidityRH = 0;

    u16sRH &= ~0x0003;

    humidityRH = (100 * (float)u16sRH / 65535);

    return humidityRH;
}

```

4. 任务创建:

创建一个线程任务 `sensor_task_entry`, 用于定期采集温湿度数据。

设置线程的属性, 如名称、优先级和栈大小。

```

/* 创建传感器数据采集任务 */
attr.name = "sensor_task_entry";
attr.priority = 25;
if (osThreadNew((osThreadFunc_t)sensor_task_entry, NULL, &attr) == NULL)
{
    printf("Falied to create sensor_task_entry!\n");
}

```

5.4. 步骤 4: 光敏传感检测设计

目标: 通过 BH1750 光敏传感器采集光照强度数据。

实现方法:

1. 传感器初始化:

初始化 I2C 接口, 设置 I2C 通信速率。

```

IoTI2cInit(IOT_I2C_IDX_1, 400000); /* baudrate: 400kbps */

```

发送初始化命令到 BH1750 传感器。

```
/* *****  
* 函数名称: Init_BH1750  
* 说 明: 写命令初始化BH1750  
* 参 数: 无  
* 返 回 值: 无  
* ***** */  
void Init_BH1750(void)  
{  
    uint8_t send_data[1] = { 0x01 };  
    IoTI2cWrite(IOT_I2C_IDX_1, (BH1750_Addr<<1)|0x00, send_data, 1);  
}
```

2. 读取 ADC 数据:

使用 IoTAdcReadChannel() 读取指定引脚的 ADC 值。

将 ADC 值转换为实际的电压值。假设 ADC 分辨率为 12 位 (即 0-4095), 参考电压为 3.3V, 则电压值为 $(adc_value * 3.3) / 4096$ 。

```
uint16_t Read_Light_Sensor(void)  
{  
    uint16_t adc_value;  
    float voltage;  
  
    adc_value =  
    IoTAdcReadChannel(LIGHT_SENSOR_ADC_PIN);  
    读取 ADC 值  
    voltage = (adc_value * 3.3) / 4096; ,  
    假设 ADC 分辨率为 12 位, 参考电压为 3.3V  
  
    return adc_value; // 返回原始 ADC 值  
}
```

3. 数据采集:

启动 BH1750 传感器进行测量。

将读取到的原始数据转换为光强度值 (单位: lux)。BH1750 的输出数据为 16 位, 需要除以 1.2 转换为实际的光强度值。

```
uint8_t recv_data[2] = { 0 };  
IoTI2cRead(IOT_I2C_IDX_1, (BH1750_Addr<<1)|0x01, recv_data, 2); // 读取传感器数据  
ReadData->Lux = (float)((((recv_data[0] << 8) + recv_data[1]) / 1.2);
```

4. 任务创建:

创建一个线程任务 sensor_task_entry, 用于定期采集光照强度数据。

设置线程的属性, 如名称、优先级和栈大小。

```

/* 创建传感器数据采集任务 */
attr.name = "sensor_task_entry";
attr.priority = 25;
if (osThreadNew((osThreadFunc_t)sensor_task_entry, NULL, &attr) == NULL)
{
    printf("Falied to create sensor_task_entry!\n");
}

```

5.5. 步骤 5: wifi 通信模块程序设计

目标: 通过 WIFI 模块连接路由器, 实现 TCP/IP 网络通信, 并将数据上报至华为 IoT 平台。

实现方法:

1. WIFI 初始化:

注册 WIFI 事件回调函数, 用于处理扫描状态、连接状态等事件。

```

printf("<-Wifi Init-->\r\n");
g_wifiEventHandler.OnWifiScanStateChanged = OnWifiScanStateChangedHandler;
g_wifiEventHandler.OnWifiConnectionChanged = OnWifiConnectionChangedHandler;
g_wifiEventHandler.OnHotspotStaJoin = OnHotspotStaJoinHandler;
g_wifiEventHandler.OnHotspotStaLeave = OnHotspotStaLeaveHandler;
g_wifiEventHandler.OnHotspotStateChanged = OnHotspotStateChangedHandler;
error = RegisterWifiEvent(&g_wifiEventHandler);

```

使能 WIFI 模块, 并判断 WIFI 是否激活。

```

//使能WIFI
if (EnableWifi() != WIFI_SUCCESS)
{
    printf("EnableWifi failed, error = %d\r\n", error);
    return -1;
}

```

2. WIFI 连接:

扫描周围的 WIFI 热点, 并获取热点列表。

```

//轮询查找WiFi列表
do{
    //重置标志位
    ssid_count = 0;
    g_staScanSuccess = 0;
    //开始扫描
    Scan();
    //等待扫描结果
    WaitSacnResult();
    //获取扫描列表
    error = GetScanInfoList(info, &size);
}while(g_staScanSuccess != 1);

```

搜索指定的 WIFI 热点，连接到热点并获取 IP 地址。

```
//连接指定的WiFi
if (ConnectTo(result) == WIFI_SUCCESS && WaitConnectResult() == 1)
{
    printf("WiFi connect succeed!\r\n");
    g_lwip_netif = netifapi_netif_find(SELECT_WLAN_PORT);
    break;
}

//启动DHCP
if (g_lwip_netif)
{
    dhcp_start(g_lwip_netif);
    printf("began to dhcp");
}
```

3. 数据上报:

使用 MQTT 协议将采集到的环境数据（温度、湿度、光照强度）上报到华为 IoT 平台。

```
/* 将数据发送到华为IoT平台 */
oc_mqtt_profile_propertyreport(NULL,&service);
```

接收平台下发的控制指令

```
connect para.rcvfunc =      msg rcv callback;
```

4. 任务创建:

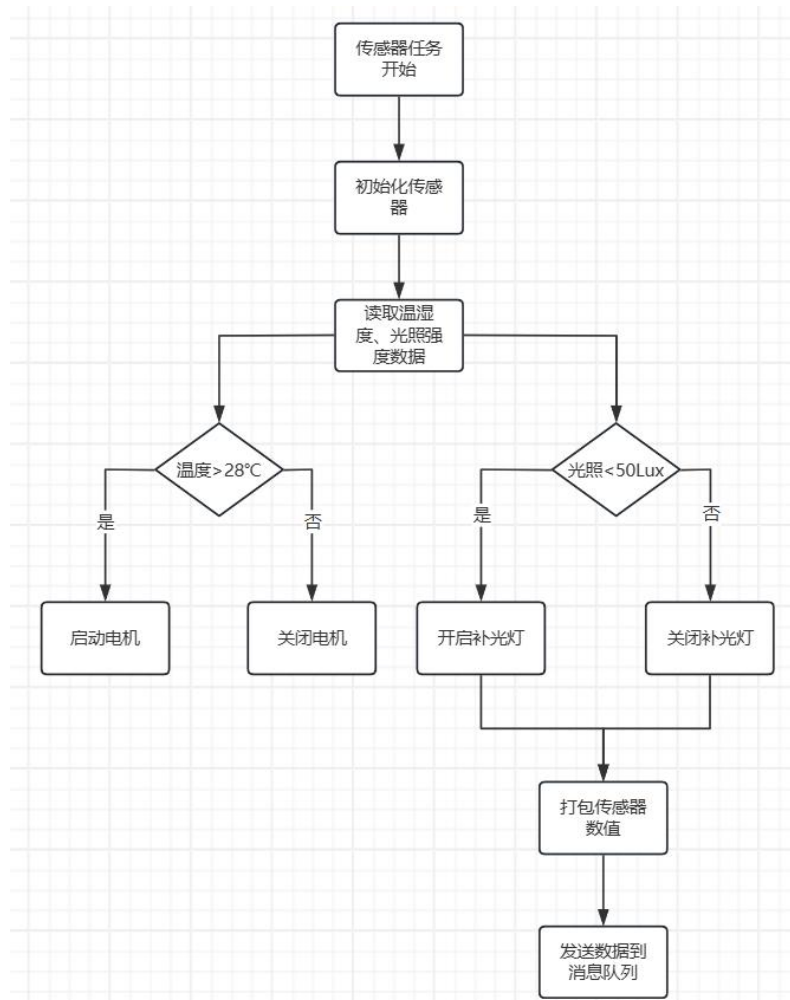
创建一个线程任务 `iot_cloud_task_entry`，用于处理 WIFI 连接和数据上报。
设置线程的属性，如名称、优先级和栈大小。

```
/* 创建端云互通任务 */
attr.name = "report_task_entry";
attr.stack_size = 1024*20;
attr.priority = 24;
if (osThreadNew((osThreadFunc_t)iot_cloud_task_entry, NULL, &attr) == NULL)
{
    printf("Falied to create report_task_entry!\n");
}
```

5.6. 步骤 6：主程序的设计流程

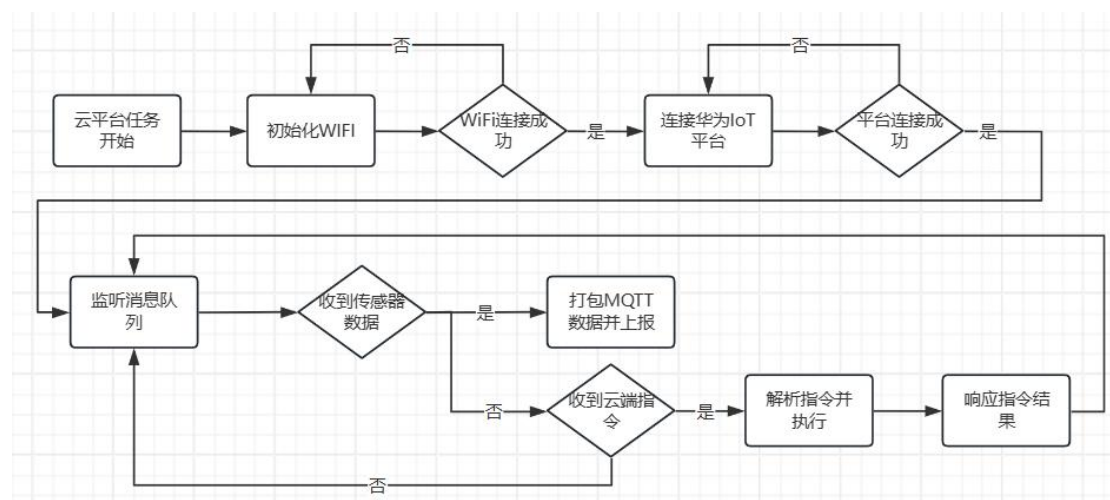
1. 传感器任务流程图

定时采集温湿度、光照数据，并根据阈值自动控制补光灯和电机。



2. 云平台任务流程

连接华为 IoT 平台，上报传感器数据并处理云端下发的控制指令。



6. 设计结果及分析

6.1. 项目实施过程

本项目基于鸿蒙操作系统（OpenHarmony）和 BearPi-HM Nano 开发板，实现了一个完整的智慧农业大棚监控系统。系统的主要功能包括：

1. 按键检测与 LED 灯控制：

通过外部中断检测按键按下事件，实现 LED 灯的状态切换（开、关、闪烁）。
按键 F1 控制 LED 灯的开启和关闭，按键 F2 控制 LED 灯的闪烁状态。

2. 环境数据采集：

使用 E53_IA1 扩展板的 SHT30 温湿度传感器和 BH1750 光敏传感器，实时监测大棚内的温度、湿度和光照强度。

采集的数据通过队列机制传递到云平台任务进行处理。

3. 设备控制：

根据光照强度自动控制补光灯的开启和关闭。

根据温度自动控制通风电机的启停。

4. 数据通信：

通过 WIFI 模块连接路由器，实现 TCP/IP 网络通信。

使用 MQTT 协议将采集到的数据上报到华为 IoT 平台，并接收平台下发的控制指令。

5. 本地交互：

通过板载按键（GPIO 中断）切换 LED 灯状态（开/关/闪烁）。

本地串口日志输出，便于调试。

6.2. 系统框架

1. 传感器任务：定时采集数据并触发本地控制逻辑。
2. 云平台任务：管理 WiFi 连接、MQTT 通信及指令响应。
3. 按键任务：实时检测用户输入。

6.3. 遇到的问题及解决方案

1. WIFI 连接问题：

问题：在连接 WIFI 时，扫描到的热点列表中没有目标热点。

解决方案：确保 WIFI 热点名称和密码正确，增加扫描超时时间，确保扫描

操作有足够的时间完成。

2. 云平台连接问题：

问题：连接华为 IoT 平台时，连接失败。

解决方案：检查设备 ID、设备密钥、服务器 IP 和端口是否正确。确保网络连接正常，增加连接超时时间。

3. 补光灯和电机控制逻辑问题：

问题：补光灯和电机的控制逻辑在某些情况下会出现误触发。

解决方案：引入迟滞逻辑，避免在阈值附近频繁切换状态。增加状态切换的延时，确保控制逻辑的稳定性。

4. 自动补光逻辑抖动问题：

问题：光照值在阈值附近波动导致灯光频繁开关

解决方案：引入迟滞逻辑，设置一个迟滞范围（如 10%的阈值范围），避免在阈值附近频繁切换状态。

5. 电机抖动问题：

问题：在自动控制电机时，当温度接近阈值时，电机会出现频繁启停的现象，导致抖动。

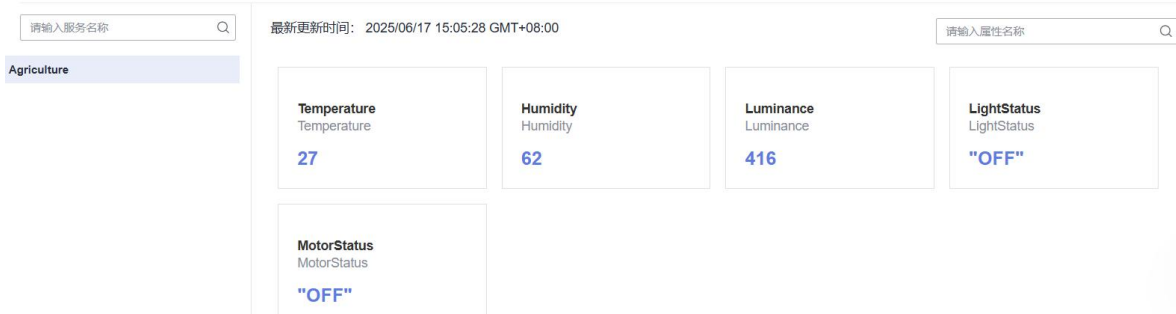
解决方案：引入迟滞逻辑，设置一个迟滞范围（如 2° C 的阈值范围），避免在阈值附近频繁切换状态。

6. 未引入头文件导致代码编译失败问题：

问题：在编写代码时，未引入必要的头文件，导致编译器无法识别某些函数或变量，编译失败。

解决方案：仔细检查代码中使用的所有函数和变量，确保对应的头文件已被正确引入。例如，使用 `IoTGpioInit` 函数时，需要确保引入了 `"iot_gpio.h"` 头文件；

6.4. 界面效果图



7. 设计结论

7.1. 设计得到的结论

通过本次课程设计，我们成功实现了一个基于鸿蒙操作系统（OpenHarmony）和 BearPi-HM Nano 开发板的智慧农业大棚监控系统。该系统具备以下主要功能和特点：

1. 功能完整性：

实现了按键检测与 LED 灯控制功能，用户可以通过按键切换 LED 灯的状态（开、关、闪烁）。

实现了环境数据采集功能，能够实时监测大棚内的温度、湿度和光照强度。

实现了设备控制功能，根据光照强度自动控制补光灯的开启和关闭，根据温度自动控制通风电机的启停。

实现了数据通信功能，通过 WIFI 模块连接到华为 IoT 平台，能够将采集到的数据上报到平台，并接收平台下发的控制指令。

实现了本地交互功能，通过串口日志输出，便于调试和监控系统运行状态。

2. 稳定性与可靠性：

通过引入迟滞逻辑和状态切换延时，解决了补光灯和电机控制逻辑的抖动问题，确保了系统的稳定性。

优化了队列机制，确保数据能够及时处理，避免数据堆积，提高了系统的响应速度。

通过增加 WIFI 连接和云平台连接的超时时间，确保了网络连接的稳定性。

3. 可扩展性：

系统设计具有良好的可扩展性，可以方便地添加更多的传感器或控制设备。

通过调整配置参数，可以轻松适配不同的 WIFI 热点和云平台。

4. 用户体验：

通过串口日志输出，用户可以实时了解系统的运行状态和传感器数据。

系统的控制逻辑简单直观，用户可以通过按键轻松切换 LED 灯的状态。

7.2. 课程设计实现的效果说明

1. 功能实现：

项目成功实现了所有设计功能，包括按键检测、LED 灯控制、环境数据采集、设备控制和数据通信。

系统能够稳定运行，实时采集和处理数据，并与华为 IoT 平台进行交互。

2. 问题解决：

在开发过程中遇到的问题（如 WIFI 连接问题、传感器数据校准问题、队列机制问题、云平台连接问题、补光灯和电机控制逻辑问题、未引入头文件导致编译失败问题、自动补光逻辑抖动问题和电机抖动问题）均得到了有效解决。

通过引入迟滞逻辑和状态切换延时，解决了补光灯和电机控制逻辑的抖动问题，确保了系统的稳定性。

3. 系统运行：

系统在实际运行中表现良好，能够稳定采集和处理环境数据，并根据预设逻辑自动控制补光灯和通风机。

通过串口日志输出，用户可以实时了解系统的运行状态和传感器数据，便于调试和监控。

4. 未来改进方向：

增加更多的传感器（如土壤湿度传感器、CO₂ 浓度传感器）以实现更全面的环境监测。

增加用户界面（如通过手机 APP 或 Web 界面）以提供更友好的交互体验。

优化数据处理算法，进一步提高系统的响应速度和准确性。

8. 问题及心得体会

8.1. 遇到的问题

1. 硬件调试问题：

在硬件连接初期，由于对开发板和传感器的引脚定义不熟悉，导致部分引脚连接错误，使得程序无法正常运行。需仔细阅读开发板和传感器的文档，核对引脚定义并重新连接硬件。在后续开发中，制作了引脚连接图，方便快速定位问题。

2. 软件编译问题：

在代码编写过程中，多次出现因未引入头文件或函数声明错误导致的编译失败。

在后续编写代码时，仔细检查每个函数和变量的使用，确保所有必要的头文件都已正确引入。同时，适当用自动补全功能，避免拼写错误。

3. 网络连接问题：

在连接 WIFI 时，程序有时无法成功连接到指定的热点，导致系统无法正常工作。增加 WIFI 扫描和连接的超时时间，确保程序有足够的时间完成连接操作。同时，优化 WIFI 连接逻辑，增加错误处理机制，确保在连接失败时能够自动重试。

4. 数据稳定性问题：

在自动补光和电机控制逻辑中，当环境数据接近阈值时，补光灯和电机出现频繁开关的现象，导致抖动。引入迟滞逻辑，设置一个迟滞范围，避免在阈值附近频繁切换状态。同时，增加状态切换的延时，确保控制逻辑的稳定性。

8.2. 心得体会

1. 理论与实践结合的重要性：

通过本次课程设计，深刻体会到理论知识与实际操作相结合的重要性。虽然在课堂上学到了许多网络协议和嵌入式系统开发的理论知识，但在实际开发过程中，仍然遇到了许多问题。通过不断查阅资料和调试代码，逐步解决了这些问题，进一步加深了对理论知识的理解。

2. 问题解决能力的提升：

在开发过程中，遇到了各种各样的问题，从硬件连接错误到软件编译失败，再到网络连接不稳定等。通过不断尝试和探索，逐步找到了解决问题的方法。这个过程不仅提升了我的问题解决能力，也让我学会了如何在面对困难时保持冷静，逐步分析问题并找到解决方案。

3. 持续学习的必要性：

在开发过程中，我发现自己在某些领域的知识还存在不足，例如网络协议的细节和嵌入式系统的优化。这让我意识到，技术是不断发展的，只有持续学习，才能跟上时代的步伐，不断提升自己的技术水平。在后续的学习中，我计划深入学习网络协议和嵌入式系统开发的高级知识，进一步提升自己的能力。

9. 对本课程设计的改进建议

9.1. 课程内容方面

1. 增加实践案例：

建议在课程中增加更多的实践案例，特别是与实际应用场景相结合的案例。通过实际案例，学生可以更好地理解理论知识的应用，并提高解决实际问题的能力。

2. 引入项目管理知识：

建议在课程中引入项目管理的基本知识，如需求分析、设计文档编写、测试计划等。通过这些知识的学习，学生可以更好地规划和管理自己的课程设计项目，提高项目的成功率。

9.2. 实验环境方面

1. 提供更多的开发板资源:

建议学校提供更多的开发板资源，方便学生进行实验和开发。

2. 优化实验指导文档:

建议优化实验指导文档，增加详细的硬件连接图和代码注释。通过这些文档，学生可以更快地理解和掌握实验内容，减少调试时间。

3. 提供在线技术支持:

建议学校提供在线技术支持平台，如论坛或问答社区，方便学生在遇到问题时能够及时获得帮助。同时，建议定期组织线上答疑活动，解答学生在实验过程中遇到的问题。

10. 参考文献

[1] 鲁维佳,果颖,潘玉恒,庞翹楚.智能家居控制系统的设计及实现[J].实验室科学,2017,20(04):31-33+37

[2] 郁有文.传感器原理及工程应用[M].西安:西安电子科技大学出版社,2014年9月

[3] 华为技术有限公司. 华为 IoT 开发指南 [M]. 深圳: 华为技术有限公司, 2023.

[4] 刘宏, 王伟. 物联网通信技术与应用 [M]. 北京: 机械工业出版社, 2021.

[5] 张建国, 李晓红, 王立新. 基于云平台的远程监控系统设计与实现[J]. 自动化技术与应用, 2018, 37(8): 45-48+53